

LAPQ lib

Abstract

This report documents lapq, an experimental C99 skip-list priority queue with learning-augmented insertion hints, Python bindings, graph-based dataset generation, and a roadmap toward machine-learning predictors.

Contents

Abstract	1
1 lapq: Learning-Augmented Priority Queue Experiments	3
2 Detailed Outline	4
3 1. Introduction and Motivation	5
4 2. Theoretical Background	6
5 3. C Library Architecture	7
6 4. Prediction Hints in the C Core	8
7 5. Difference from the Paper	9
8 6. Testing Strategy	10
9 7. C Benchmarks	11
10 8. Python Binding	12
11 9. Graph Experiment Layer	13
12 10. Machine Learning Roadmap	14
13 11. Algorithmic Roadmap	15
14 12. Packaging and Reproducibility	16
15 13. Current Status	17
16 14. Conclusion	18
17 15. References	19

1 lapq: Learning-Augmented Priority Queue Experiments

This notebook is the living project report. It documents the C library, the Python experimental layer, the relationship with the reference paper, and the roadmap toward machine-learning predictors. Sections marked as work in progress are intentionally left as scaffolding to be completed as the experiments mature.

2 Detailed Outline

1. Introduction and Motivation
 1. Project context
 2. Learning-augmented priority queues
 3. Why skip lists
 4. Scope of this repository
2. Theoretical Background
 1. Standard priority queues
 2. Skip lists
 3. Learning-augmented algorithms
 4. The LAPQ model from the reference paper
 5. Clean comparisons, dirty comparisons, and predictions
3. C Library Architecture
 1. Public API
 2. Opaque queue object
 3. Caller-owned items
 4. Skip-list backend
 5. Handle table
 6. Statistics and instrumentation
 7. Error handling and invariants
4. Prediction Hints in the C Core
 1. Hint representation
 2. No-hint baseline
 3. Predecessor hints
 4. Invalid, stale, and cross-queue hints
 5. Backward correction
 6. Bottom-up expansion
 7. Top-down refinement
 8. Rank hints and current limitations
5. Difference from the Paper
 1. What is implemented
 2. What is inspired by Algorithm 2
 3. What is not implemented yet
 4. Why heaps are not the chosen target
 5. Current theoretical claims and non-claims
6. Testing Strategy
7. C Benchmarks
8. Python Binding
9. Graph Experiment Layer
10. Machine Learning Roadmap
11. Algorithmic Roadmap
12. Packaging and Reproducibility
13. Current Status
14. Conclusion
15. References

3 1. Introduction and Motivation

`lapq` is an experimental project with two deliberately separated goals. First, it implements a compact C99 priority queue based on skip lists and prediction hints. Second, it provides a Python layer for datasets, graph experiments, synthetic predictors, and eventually machine-learning models.

Predictions never replace correctness-critical comparisons. They are passed to the C core as hints, and the C core validates and corrects them using the ordinary comparator.

4 2. Theoretical Background

This section will summarize the required background: priority queues, skip lists, learning-augmented algorithms, and the model described in the learning-augmented priority queue paper [BC24].

Work in progress. The final report should clearly distinguish clean comparisons, dirty comparisons, pointer predictions, and rank predictions.

5 3. C Library Architecture

The public API lives in `include/lapq/lapq.h` and keeps `struct lapq` opaque. The implementation is split into focused modules: `src/lapq.c` for the public facade, `src/skiplist.c` for skip-list logic and hinted search, `src/handles.c` for generational handles, `src/stats.c` for instrumentation, and `src/lapq_internal.h` for private shared structures.

The C core stores caller-owned `void *` items. It does not know how predictions are computed and does not own any machine-learning logic.

6 4. Prediction Hints in the C Core

The learning-augmented interface is `struct lapq_hint`. The current C core supports no hint, predecessor hints, and experimental rank hints. Predecessor hints are validated through generational handles. If valid, the search starts near the predicted predecessor; if invalid, the implementation falls back to ordinary search and increments `invalid_hints`.

The hinted search has three explicit correction phases: backward correction, bottom-up expansion, and top-down refinement. These phases are also reflected in the instrumentation counters.

7 5. Difference from the Paper

The implementation is inspired by the pointer-prediction search model, but it does not yet implement the full theoretical system from the reference paper. In particular, dirty comparisons are not present, rank hints are not backed by an asymptotically strong indexed structure, and the repository currently has one concrete skip-list backend.

This section will be expanded with precise claims and non-claims once the experimental results are stable.

8 6. Testing Strategy

The C tests cover insertion and extraction order, duplicate keys, handles, `decrease_key`, arbitrary removal, invalid hints, stale handles, hinted search phases, and a randomized oracle. Sanitizer builds are run through `make check`.

The Python tests cover the CPython binding, handle-returning insertions, hint passing, graph loading, Dijkstra baselines, dataset generation, and CLI paths.

9 7. C Benchmarks

The C benchmark compares baseline insertions with perfect, noisy, and bad predecessor hints, plus a dedicated `decrease_key` workload. It reports time, clean comparisons, level-0 steps, express-level steps, per-phase traversal counters, hint counts, invalid hints, average error, maximum error, and a checksum.

Work in progress. This section should eventually include tables and plots from reproducible benchmark runs.

10 8. Python Binding

The Python package exposes `PriorityQueue` and `Handle` through a CPython extension. The binding maps insertion, extraction, peeking, removal, stats, invariant checks, and predecessor/rank hint insertion to the C core.

Python-facing `decrease_key` is intentionally not exposed yet. The binding needs an explicit policy for mutating the priority stored inside C-owned queue items before calling the C `lapq_decrease_key` operation.

11 9. Graph Experiment Layer

The graph layer loads DIMACS shortest-path graphs into a compact CSR representation. It provides Dijkstra baselines with Python `heapq`, Dijkstra with `lapq`, and Dijkstra with synthetic LAPQ hints. Dataset utilities emit CSV rows for priority-queue insertion events, including Python-computed oracle predecessor information.

The road graph files under `graphs/dimacs/` are local data and are intentionally kept out of Git.

12 10. Machine Learning Roadmap

Real ML models are intentionally out of scope until the dataset format and baseline experiments are stable. Candidate targets include predecessor prediction, rank prediction, or a hybrid policy. Candidate features may come from Dijkstra state, graph topology, current distance values, queue size, and edge weights.

Work in progress. This section will define feature sets, training splits, evaluation metrics, and model choices.

13 11. Algorithmic Roadmap

The C core is complete for v0.1, but not complete with respect to all ideas in the reference paper. Future algorithmic work is deliberately staged:

- dirty comparisons;
- stronger rank hints;
- indexed skip lists or vEB-like auxiliary structures;
- backend abstraction once multiple concrete backends exist.

These are roadmap items, not current bugs. The criteria for changing the C core should come from measured Python experiments and clear theoretical goals.

14 12. Packaging and Reproducibility

The project uses `setuptools` and a CPython extension. GitHub Actions runs tests and builds wheels for Linux, macOS, and Windows. Doxygen API documentation can be generated locally with `make api-docs`; the generated HTML is ignored by Git.

Release artifacts are built from Git tags and attached to GitHub Releases.

15 13. Current Status

Completed components include the C skip-list priority queue, handles, predecessor hints, experimental rank hints, instrumentation, C benchmarks, Python bindings, DIMACS/CSR graph loading, Dijkstra baselines, synthetic hint scenarios, dataset CSV generation, packaging, CI, and Doxygen infrastructure.

Current limitations include the absence of dirty comparisons, strong rank-aware structures, Python-facing `decrease_key`, and real machine-learning models.

16 14. Conclusion

The repository currently provides a stable experimental foundation: a compact C core with clean correctness semantics and a Python layer designed to generate and evaluate predictions. The next research step is empirical: collect datasets, compare synthetic hint policies, and use those results to decide when ML models and deeper C changes are justified.

17 15. References

Bibliography

- [BC24] Ziyad Benomar and Christian Coester. Learning-augmented priority queues. In *Advances in Neural Information Processing Systems 38*, 2024. Local copy: [docs/papers/learning_augmented_priority_queues.pdf](#).